AutoSystemDesign

От бизнес-требований к проверяемой модели архитектуры микросервисной системы

Дичковский Владимир

Какую задачу решает инструмент

Проблема

- Архитектор тратит время на ручной перевод требований в диаграммы компонентов.
- Диаграммы компонентов, диаграммы последовательностей и API необходимо согласовывать между собой.

Решение

- LLM формирует структурированную модель компонентов и их взаимодействий.
- Детерминированные валидаторы проверяют типы, связи и ограничения модели.
- На её основе строятся артефакты (C4, диаграммы последовательностей, OpenAPI, отчёт производительности).

Вход: фрагмент BRD интернет-магазина

Раздел «Оформление заказа»

При оформлении заказа система проверяет наличие товара. После успешной оплаты заказ подтверждается. Если оплата отклонена, заказ отменяется.

Раздел «Комплектация»

Заказы с отклонённой оплатой передаются на склад для комплектации.

Раздел «Нагрузка и качество»

Обычная нагрузка — 120 запросов в секунду. В пике система должна обрабатывать 400 запросов в секунду. p95 — не выше 300 мс; доступность — 99,9%.

Извлечение: текст BRD → требование

Функциональное требование

«При оформлении заказа система проверяет наличие товара.»

⟹

FR-01
Проверять наличие товара при оформлении заказа.

Нефункциональное требование

«В пике система должна обрабатывать 400 запросов в секунду.»

⟹

NFR-01
Пиковая нагрузка: 400 запросов/с.

Проверка требований: пример противоречия

Раздел «Оформление заказа»

«Если оплата отклонена, заказ отменяется.»

Раздел «Комплектация»

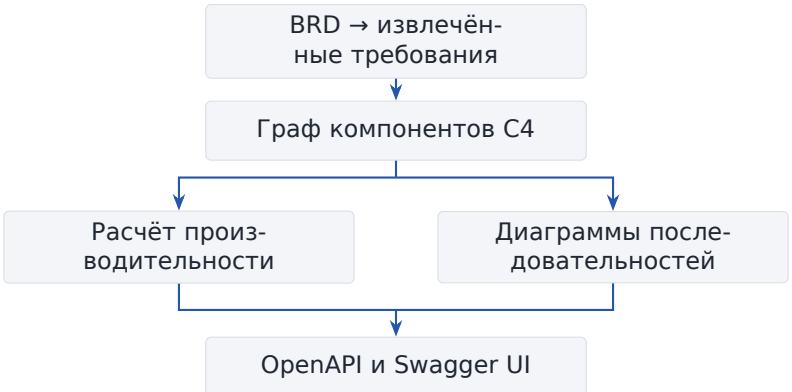
«Заказы с отклонённой оплатой передаются на склад для комплектации.»

Конфликт требований

При отклонённой оплате заказ должен одновременно отменяться и передаваться на склад. Противоречие сохраняется для анализа человеком.

Каждое извлечённое требование связывается с цитатой BRD. Цитаты сверяются с исходным текстом.

Архитектура конвейера и зависимости этапов



Генерация архитектурной модели и исправление ошибок

Генерация

- LLM возвращает структурированное описание компонентов, связей и взаимодействий.
- Из проверенной модели программно строятся диаграммы и спецификации.

Проверка и повторная генерация

- Валидаторы проверяют обязательные поля, типы и связи.
- При ошибке LLM получает перечень нарушений и исправляет модель.
- Количество повторных попыток ограничено.

Модель компонентов: зачем нужны типы и параметры

Типы → проверка связей

- Узел помечается типом: клиент, сервис, база данных, кэш, брокер.
- Тип определяет, какие связи между узлами допустимы.
- Например: клиент → API → база данных.

Параметры → расчёты

- Число реплик, время обработки запроса и размер очереди.
- По ним оцениваются пропускная способность, загрузка и ожидание.

Одна модель используется для построения диаграммы компонентов и проверки нагрузки на архитектуру.

Валидаторы графа компонентов: три группы проверок

Структура

Уникальные ID, допустимые связи, отсутствие циклов

Семантика

Поставщик и потребитель брокера, путь мимо кэша

Расчёты

Поток RPS, пропускная способность, насыщение

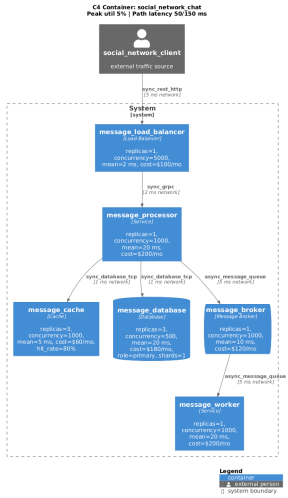
Допустимо

Клиент → API → БД

Недопустимо

Клиент → БД напрямую

Пример: диаграмма компонентов C4

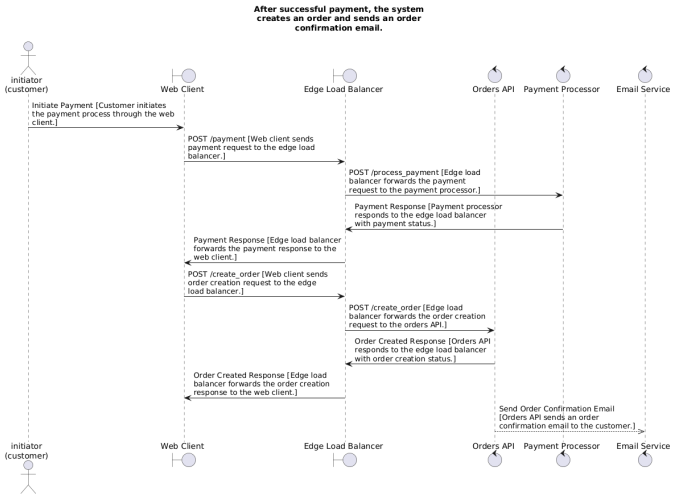


Проверка диаграмм последовательностей

Проверяемые инварианты

1. Идентификатор требования на диаграмме должен совпадать с требованием, для которого она построена.
2. Каждый компонент на диаграмме должен ссылаться на существующий узел графа архитектуры.
3. Сообщения-события и уведомления должны иметь асинхронный режим доставки.

Пример: диаграмма последовательностей



REST API: контракт и пример OpenAPI JSON

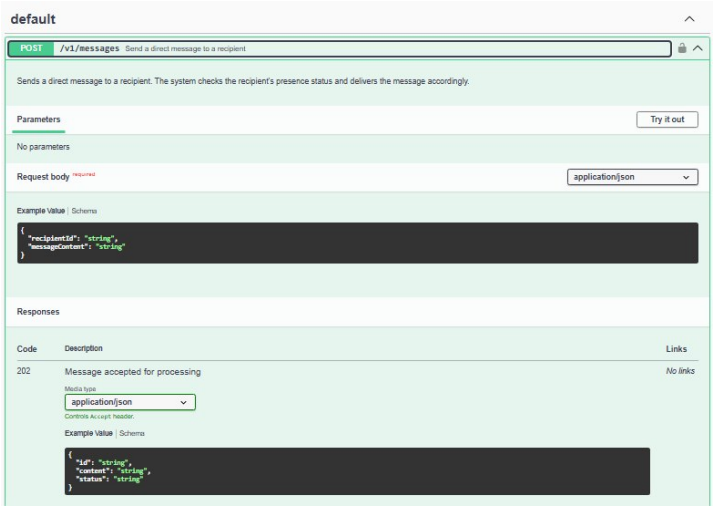
Что получаем

- Методы и маршруты по функциональным требованиям.
- Структуры запросов и ответов.
- Проверка уникальности операций и отсутствия конфликтующих маршрутов.

Фрагмент OpenAPI JSON

```
{
  "openapi": "3.1.0",
  "paths": {
    "/orders": {
      "post": {
        "operationId": "createOrder",
        "responses": {
          "201": {"description": "Created"},
          "409": {"description": "Out of stock"}
        }
      }
    }
  }
}
```

Пример: Swagger UI



Проверка производительности: теория массового обслуживания

Входные параметры

- Интенсивность запросов (RPS).
- Количество реплик компонентов.
- Среднее время обработки и параметры очередей.

Результаты расчётов

- Распределение потока запросов по графу.
- Пропускная способность и загрузка компонентов.
- Оценка ожидания в очередях и среднего времени ответа.

Пример: отчёт анализа производительности

Анализ производительности

Пример: checkout_baseline

Синхронный путь с наибольшей задержкой

Максимальная оценка средней задержки пути: 32.10 мс

Критический путь: client → checkout_api → payment_service

Показатели компонентов

Компонент	Нагрузка запр./с	Пропускная запр./с	Загрузка %	Ожидание мс	Ответ мс
checkout_api	60.00	500.00	12.0	0.02	8.02
inventory_db	60.00	333.33	18.0	0.09	12.09
payment_service	60.00	150.00	40.0	2.08	22.08

Пример

60 RPS

нагрузка узлов

32,10 мс

максимальная оценка средней задержки пути